

LAB 7
RUBAISHA ARIF
CT-25067

CT-260 OOP(P)
Spring'26
Section B

DYNAMIC ARRAY EXAMPLE 1

```
main.cpp
1 #include <iostream>
2 using namespace std;
3 class Array {
4     int size;
5     int *ptr
6 public:
7     Array(int s = 0) : size(s), ptr(nullptr) {
8         if (size > 0)
9             ptr = new int[size];
10        if (ptr == nullptr)
11            return;
12        Array(Array& ob) {
13            if (ob.size > 0) {
14                size = ob.size;
15                ptr = new int[size];
16                if (ptr == nullptr)
17                    return;
18                for (int i = 0; i < size; i++) {
19                    ptr[i] = ob[i];
20                }
21            } else
22                return;
23        }
24        int& operator[](int in) {
25            if (in >= size) {
26                cout << "index " << in << " is out of bound" << endl;
27                return ptr[0];
28            }
29            return ptr[in];
30        }
31        Array operator+(Array& ob) {
32            if (size == ob.size) {
33                Array temp(size);
34                for (int i = 0; i < size; i++)
```

0
2
4
6
8
3
4
5
6
7
3 6 9 12 15
120
--- Code Execution Successful ---

```
main.cpp
31        Array temp(size);
32        for (int i = 0; i < size; i++)
33            temp[i] = ptr[i] + ob[i];
34        return temp;
35    } else { cout << "The size of array does not match!" << endl;
36    }
37    return 0; }
38    int operator*(Array& ob) {
39        if (size == ob.size) {
40            int temp = 0;
41            for (int i = 0; i < size; i++)
42                temp += ptr[i] * ob[i];
43            return temp;
44        } else {
45            cout << "The size of array does not match!" << endl;
46            return 0;
47        }
48    }
49    ~Array() {
50        delete[] ptr;
51    }
52    int main() {
53        Array a1(5), a2(5);
54        for (int i = 0; i < 5; i++) {
55            a1[i] = i * 2;
56            a2[i] = i + 3;
57        }
58        for (int i = 0; i < 5; i++)
59            cout << a1[i] << endl;
60        for (int i = 0; i < 5; i++)
61            cout << a2[i] << endl;
62        Array a3(a1);
63        Array a4 = a1 + a2;
64        for (int i = 0; i < 5; i++)
65            cout << a4[i] << "\t";
66        cout << endl;
67        cout << a1 * a2 << endl;
68        return 0;
69    }
```

0
2
4
6
8
3
4
5
6
7
3 6 9 12 15
120
--- Code Execution Successful ---

FRIEND FUNCTION EXAMPLE 1

main.cpp	Run	Output
<pre>1 #include<iostream> 2 using namespace std; 3 class beta; 4 class alpha{ 5 int data; 6 public: 7 alpha(): data(3) { } 8 friend int frifunc(alpha, beta); 9 }; 10 11 class beta{ 12 int data; 13 public: 14 beta(): data(7) { } 15 friend int frifunc(alpha, beta); 16 }; 17 18 int frifunc(alpha a, beta b){ 19 return(a.data+b.data); 20 } 21 22 int main(){ 23 alpha aa; 24 beta bb; 25 cout << frifunc(aa, bb) << endl; 26 return 0; 27 }</pre>		<pre>10 === Code Execution Successful ===</pre>

FRIEND FUNCTION EXAMPLE 2

```
main.cpp
1 #include <iostream>
2 using namespace std;
3 class Array {
4     int size;
5     int *ptr;
6 public:
7     Array(int s = 0) : size(s), ptr(nullptr) {
8         if (size > 0)
9             ptr = new int[size];
10        if (ptr == nullptr)
11            return;
12        Array(Array& ob) {
13            if (ob.size > 0) {
14                size = ob.size;
15                ptr = new int[size];
16                if (ptr == nullptr)
17                    return;
18                for (int i = 0; i < size; i++) {
19                    ptr[i] = ob[i];
20                }
21            }
22            else
23                return;
24        }
25        int& operator[](int in) {
26            if (in >= size) {
27                cout << "index " << in << " is out of bound" << endl;
28                return ptr[0];
29            }
30            return ptr[in];
31        }
32        Array operator+(Array& ob) {
33            if (size == ob.size) {
34                Array temp(size);
35                for (int i = 0; i < size; i++)
```

```
0
2
4
6
8
3
4
5
6
7
3 6 9 12 15
120

--- Code Execution Successful ---
```

FRIEND CLASS EXAMPLE 1

```
main.cpp
1 #include<iostream>
2 using namespace std;
3
4 class alpha{
5 private:
6     int data1;
7 public:
8     alpha( ): data1(99) { }
9     friend class beta;
10 };
11
12 class beta{
13 public:
14     void func1(alpha a) { cout << "\ndata1=" << a.data1; }
15     void func2(alpha a) { cout << "\ndata1=" << a.data1; }
16 };
17
18 int main( ){
19     alpha a;
20     beta b;
21     b.func1(a);
22     b.func2(a);
23     cout << endl;
24     return 0;
25 }
```

```
data1=99
data1=99

=== Code Execution Successful ===
```

EXERCISE Q1

1. Create a class for 2D dynamic Array, of type integer, by using a double pointer, that is, a pointer to the array of pointers. The class must have the following methods.

- Default, parameterized, and copy constructor
- Overloaded assignment operator
- Overloaded +, -, and * operator
- Overloaded indexing operator
- Destructor.

Write a test program for verifying the working of the designed class.

```
1 //q1
2 #include <iostream>
3 using namespace std;
4
5 class DArray {
6     int** array;
7     int row, col;
8 public:
9     DArray()
10    {
11        row = 0;
12        col = 0;
13        array = nullptr;
14        cout << "Default constructor \n";
15    }
16    DArray(int r, int c) {
17        row = r;
18        col = c;
19
20        array = new int*[row];
21        for (int i = 0; i < row; i++)
22            array[i] = new int[col];
23
24        for (int i = 0; i < row; i++)
25            for (int j = 0; j < col; j++)
26                array[i][j] = 0;
27
28        cout << "Parameterized constructor\n";
29    }
30    DArray(const DArray &other) {
31        row = other.row;
32        col = other.col;
33
34        array = new int*[row];
35        for (int i = 0; i < row; i++){
36            array[i] = new int[col];
37            for (int j = 0; j < col; j++)
38                array[i][j] = other.array[i][j];
39
40        cout << "Copy constructor \n";
41    }
42
43    ~DArray() {
44        if (array != nullptr) {
45            for (int i = 0; i < row; i++)
46                delete[] array[i];
47            delete[] array;
48        }
49        cout << "Destructor called\n";
50    }
51
52    DArray &operator=(const DArray &other) {
53        if (this == &other)
54            return *this;
55
56        // free old memory
57        for (int i = 0; i < row; i++)
58            delete[] array[i];
```

```
Parameterized constructor
Matrix A:
1 2
3 4
Copy constructor
Matrix B (copy of A):
1 2
3 4
Parameterized constructor
Assignment operator called
Matrix C (assigned from A):
1 2
3 4
Parameterized constructor
Copy constructor
Destructor called
A + B:
2 4
6 8
Parameterized constructor
Copy constructor
Destructor called
A - B:
0 0
0 0
Parameterized constructor
Copy constructor
Destructor called
A * B:
7 10
15 22
A[0][1] = 2
A[1][0] = 3
Destructor called
Destructor called
Destructor called
Destructor called
Destructor called
```

=== Code Execution Successful ===

EXERCISE Q2

2. By using the 2D Array class that you created in Problem 1, solve the following problem. Given a

2D Array (matrix) of integers of size $m \times n$ with the following two properties:

- Each row is sorted in non-decreasing order
- The first integer of each row is greater than the last integer of the previous row and an integer target, return true if the target is in the matrix otherwise false otherwise.

Hint: Apply binary search algorithm.

```

1 #include <iostream>
2 using namespace std;
3
4
5 class DArray {
6     int** array;
7     int row, col;
8 public:
9     DArray()
10    {
11        row = 0;
12        col = 0;
13        array = nullptr;
14        cout << "Default constructor \n";
15    }
16    DArray(int r, int c) {
17        row = r;
18        col = c;
19
20        array = new int*[row];
21        for (int i = 0; i < row; i++)
22            array[i] = new int[col];
23
24        for (int i = 0; i < row; i++)
25            for (int j = 0; j < col; j++)
26                array[i][j] = 0;
27
28        cout << "Parameterized constructor\n"; }
29    DArray(const DArray &other) {
30        row = other.row;
31        col = other.col;
32
33        array = new int*[row];
34        for (int i = 0; i < row; i++){
35            array[i] = new int[col];
36            for (int j = 0; j < col; j++)
37                array[i][j] = other.array[i][j];
38
39            cout << "Copy constructor \n"; }
40    ~DArray() {
41        if (array != nullptr) {
42            for (int i = 0; i < row; i++)
43                delete[] array[i];
44            delete[] array;
45        }
46        cout << "Destructor called\n"; }
47    DArray &operator=(const DArray &other) {
48        if (this == &other)
49            return *this;
50        // free old memory
51        for (int i = 0; i < row; i++)
52            delete[] array[i];
53        delete[] array;
54        row = other.row;
55        col = other.col;
56        array = new int*[row];
57        for (int i=0; i<row; i++) {
58            array[i] = new int[col];

```



```

112 // Binary search
113 bool search(int target) {
114     if (row == 0 || col == 0) return false;
115     int start = 0, end = rowcol - 1;
116     while (start<= end) {
117         int mid = (start+end)/2;
118         int value = array[mid][col][mid][col];
119         if (value==target)
120             return true;
121         else if (value<target)
122             start = mid + 1;
123         else
124             end = mid - 1;
125     }
126     return false; }
127
128 int main() {
129     DArray A(2, 2);
130     A[0][0]=1; A[0][1]=2;
131     A[1][0]=3; A[1][1]=4;
132     A.print("Matrix A");
133     DArray B(A);
134     B.print("Matrix B (copy of A)");
135     DArray C(2, 2);
136     C = A;
137     C.print("Matrix C (assigned from A)");
138     DArray D = A + B;
139     D.print("A + B");
140     DArray E = A - B;
141     E.print("A - B");
142     DArray F = A * B;
143     F.print("A * B");
144     cout << "A[0][0] = " << A[0][0] << "\n";
145     cout << "A[1][0] = " << A[1][0] << "\n";
146     DArray Msearch(3,4);
147     int target = 3;
148     int values[3][4] = {
149         { 1, 3, 5, 7 },
150         {10, 11, 16, 20},
151         {23, 30, 34, 60} };
152     for (int i=0; i<3; i++) {
153         for (int j=0; j<4; j++) {
154             Msearch[i][j] = values[i][j]; } }
155     cout << "Matrix (for binary search):" << endl;
156     Msearch.print();
157     int t1 = 3;
158     int t2 =13;
159     cout<<"search for t1 \n";
160     if (Msearch.search(t1)) {
161         cout << "Output: true" << endl;
162     } else {
163         cout << "Output: false" << endl;
164     }
165     cout<<"search for t2 \n";
166     if (Msearch.search(t2)) {
167         cout << "Output: true" << endl;
168     } else {
169         cout << "Output: false" << endl;
170     }
171     return 0; }

```

To exit full screen, press and hold Esc

```

1 2
3 4
Copy constructor
Matrix B (copy of A):
1 2
3 4
Parameterized constructor
Assignment operator called
Matrix C (assigned from A):
1 2
3 4
Parameterized constructor
Copy constructor
Destructor called
A + B:
2 4
6 8
Parameterized constructor
Copy constructor
Destructor called
A - B:
0 0
0 0
Parameterized constructor
Copy constructor
Destructor called
A * B:
7 10
15 22

A[0][1] = 2
A[1][0] = 3
Parameterized constructor
Matrix (for binary search):
matrix:
1 3 5 7
10 11 16 20
23 30 34 60
search for t1
Output: true
search for t2
Output: false
Destructor called
Destructor called
Destructor called
Destructor called
Destructor called
Destructor called

```

=== Code Execution Successful ===

EXERCISE Q3

3. A company has decided to update the salaries of the employees for which updation is required in the payroll system so that employees can be paid according to the revised budget. How can you implement the concept of friend class here? Consider there are 2 classes. One is "Employee" having private data members (name, id, designation, salary etc.) and the other is "Payroll". The function for updating salaries can be made inside the "Payroll" class that can access the private member "salary" of the "Employee" class and allow the required updation. Implement this scenario.

main.cpp	Output
<pre>1 //q3 2 3 #include <iostream> 4 #include <string> 5 using namespace std; 6 7 class Employee { 8 private: 9 string name; 10 int id; 11 string designation; 12 double salary; 13 public: 14 Employee(string n, int i, string d, double s) 15 { name = n; 16 id = i; 17 designation = d; 18 salary = s; } 19 20 void display() const { 21 cout << "Name: " << name << endl; 22 cout << " ID: " << id << endl; 23 cout << " Designation: " << designation << endl; 24 cout << " Salary: " << salary << endl; 25 } 26 friend class Payroll; 27 }; 28 29 class Payroll { 30 public: 31 void updateSal(Employee& emp, double newSal) { 32 cout << "Updating salary for Employee ID: " << emp.id << endl; 33 emp.salary = newSal; 34 cout << "new salary" << emp.salary << endl; 35 } 36 }; 37 38 39 40 int main() { 41 Employee e1("ali", 100, "Engineer", 55000.0); 42 cout << "Before Update:" << endl; 43 e1.display(); 44 Payroll p; 45 p.updateSal(e1, 53000.0); 46 47 cout << "\nAfter Update: " << endl; 48 e1.display(); 49 50 return 0; 51 }</pre>	<pre>Before Update: Name: ali ID: 100 Designation: Engineer Salary: 55000 Updating salary for Employee ID: 100 new salary53000 After Update: Name: ali ID: 100 Designation: Engineer Salary: 53000 === Code Execution Successful ===</pre>

EXERCISE Q4

4. Repeat Q3 and implement this scenario using the friend function.

main.cpp	Output
<pre>1 //q4 2 #include <iostream> 3 #include <cstring> 4 using namespace std; 5 6 class Employee { 7 private: 8 string name; 9 int id; 10 string designation; 11 double salary; 12 public: 13 Employee(string n, int i, string d, double s) 14 { name = n; 15 id = i; 16 designation = d; 17 salary = s; } 18 19 void display() const { 20 cout << "Name: " << name << endl; 21 cout << " ID: " << id << endl; 22 cout << " Designation: " << designation << endl; 23 cout << " Salary: " << salary << endl; 24 } 25 friend void updateSal(Employee& emp, double newSal) ; 26 27 }; 28 void updateSal(Employee& emp, double newSal) 29 { 30 cout << "Updating salary for Employee ID: " << emp.id << endl; 31 emp.salary = newSal; 32 // cout << "new salary" << emp.salary << endl; 33 } 34 35 int main() { 36 Employee e1("ali", 100, "Engineer", 55000.0); 37 cout << "Before Update:" << endl; 38 e1.display(); 39 updateSal(e1, 60000); 40 41 cout << "\nAfter Update: " << endl; 42 e1.display(); 43 44 return 0; 45 }</pre>	<pre>Before Update: Name: ali ID: 100 Designation: Engineer Salary: 55000 Updating salary for Employee ID: 100 After Update: Name: ali ID: 100 Designation: Engineer Salary: 60000 === Code Execution Successful ===</pre>